

Sampling the residual-based merit in one pass: a generated line-search kernel whose cost is independent of the number of trial step lengths

Patrick Zulian*

September 20, 2026

Abstract

A globalized Newton solver evaluates a merit function at several trial step lengths per iteration. When the merit is the squared residual norm $\frac{1}{2}\|R(x + \alpha h)\|^2$ — the only merit available to a formulation that is not the gradient of a potential — the obvious implementation assembles the whole residual once per trial step, so a search over m step lengths costs m assemblies. Measured on a Mooney–Rivlin Kelvin–Voigt solid at 512,000 elements on one Grace node at 72 threads, $m = 12$ costs $11.8\times$ one assembly, and only 6% of a merit evaluation is independent of the step length.

We reorganize that evaluation in two steps. First, a material written as several units — here a stored energy for the elastic response and a weak residual for the viscous one — has no single form to contract, because a squared nodal residual may not be taken until every unit has contributed. Both units are 1-forms on the same fields, so they sum, and the material’s whole residual becomes one kernel: one traversal in place of two, worth $1.57\times$ the element rate and 36% of the time on the same node. Second, the m trial steps share everything upstream of the constitutive law, and what they share can be hoisted out of the trial-step loop exactly rather than approximately, because the state enters the element kernel through a quantity that is affine in α .

The resulting kernel produces all m merit values in one pass and materializes no per-step residual vector — indeed no accumulator array at all. It works one node at a time, so that node’s value is complete once its incident elements have been summed and can be squared in registers; the SIMD lanes are filled from within that one node’s patch, first by its incident elements and then by the trial steps — two sequential vector loops, never nested, the first precomputing everything the second would otherwise recompute. Normalizing each element’s orientation so the node is always the same local node makes its basis function the same in every lane and at every step, which is what allows those terms to leave both loops. The second loop’s lanes are the trial steps, because m is a quantity the line search chooses and is therefore set to the hardware’s vector length — unlike the number of elements incident on a node, which the mesh fixes. It is the only arrangement built. What this is worth is bounded by the constitutive evaluation, which repeats per trial step by construction. The same reorganization is already in place for the energy merit, and measured on the same node it costs $5.7\times$ one step at $m = 12$ for a Mooney–Rivlin-class material and $3.7\times$ for linear elasticity: the step-independent fraction rises from 6% to 57%. That is the quantity this work moves, and we report it in preference to the ratio, because a material whose stress evaluation dominates pays for every step it samples and no arrangement of the loops can make the cost flat.

*Affiliation to be completed.

1 Notation

Let $R : \mathbb{R}^n \rightarrow \mathbb{R}^n$ be the assembled nonlinear residual of the discrete problem, $x \in \mathbb{R}^n$ the current iterate and $h \in \mathbb{R}^n$ the search direction. A line search asks for the merit

$$\phi(\alpha) = \frac{1}{2} \|R(x + \alpha h)\|_2^2 = \frac{1}{2} \sum_{i=1}^n R_i(x + \alpha h)^2 \quad (1)$$

at a prescribed set of trial step lengths $\{\alpha_k\}_{k=1}^m$, with $m = 12$ for `hyperelasticity_bdf2`, $m = 13$ for `MaMAL` and $m = 20$ by default for `bench_hyperelasticity`.

The residual is a sum over the operators registered on the `Function`,

$$R(y) = \sum_{p \in \mathcal{P}} R_p(y), \quad (2)$$

and each operator assembles element-wise from a local kernel r^e ,

$$R_p(y) = \sum_{e \in \mathcal{E}_p} P_e^\top r_p^e(P_e y), \quad (3)$$

where $P_e \in \mathbb{R}^{N_s N_c \times n}$ is the gather of element e 's N_s nodes and N_c field components.

2 What is not additive, and what is

The energy merit and the residual merit differ in exactly one respect, and it governs the whole design. An energy or potential is a sum of scalars, so it is additive over operators and over elements:

$$E(y) = \sum_p E_p(y) = \sum_p \sum_e E_p^e(P_e y). \quad (4)$$

Each element contributes a number and the numbers are summed. The residual merit is not of that form. Squaring happens *after* the sum over operators and after the sum over elements, and squaring does not commute with either:

$$\frac{1}{2} \left\| \sum_p R_p \right\|^2 \neq \sum_p \frac{1}{2} \|R_p\|^2, \quad \frac{1}{2} \left\| \sum_e P_e^\top r^e \right\|^2 \neq \sum_e \frac{1}{2} \|r^e\|^2. \quad (5)$$

Equation (5) is why (1) cannot be evaluated by an element-local reduction the way an energy can, and why no operator can compute its own share of it. A node's residual is a sum over every element incident on that node, and the square may be taken only once that sum is complete.

The consequence for the interface is that *one* operator performs the contraction, and it must be handed everything the other operators contribute. We therefore split $\mathcal{P}$ into the operator M whose residual moves with the state and the remainder $\mathcal{A} = \mathcal{P} \setminus \{M\}$ whose residuals do not:

$$R(x + \alpha h) = \underbrace{\sum_{a \in \mathcal{A}} R_a}_g + R_M(x + \alpha h), \quad \frac{\partial R_a}{\partial y} \equiv 0 \quad \forall a \in \mathcal{A}. \quad (6)$$

The vector $g \in \mathbb{R}^n$ — the *accumulator* — collects the Neumann tractions, the body force and the constraint contributions, signs included, exactly as `Function::gradient` builds them. It is assembled once per Newton step rather than m times, which is already correct today only by accident: those operators are re-assembled per trial step and return the same numbers. With (6) the merit becomes

$$\phi(\alpha_k) = \frac{1}{2} \sum_{i=1}^n \left(g_i + [R_M(x + \alpha_k h)]_i \right)^2 \quad (7)$$

and the single operator M owns its evaluation for all k at once.

3 One material, several units, one form

Equation (7) assumes a single operator M whose kernel produces R_M . A material need not be written that way. Mooney–Rivlin Kelvin–Voigt is two *units* — a stored-energy density for the elastic response and a weak residual for the viscous one — and each publishes its own kernels, which the operator runs one after the other into the same vector. Neither can contract the merit, for the reason of (5) applied one level up: after the first traversal no node is finished, so nothing may be squared.

The resolution is not to fuse two kernels in one traversal, which would mean carrying two element-kernel calling conventions through a single loop nest. It is that both units are 1-forms on the same fields, and 1-forms add. Writing $c_i^{(u)}$ and $\mathbf{c}_i^{(u)}$ for the coefficients unit u contracts against the test value and the test gradient of component i ,

$$R_i(v) = \sum_u \left(c_i^{(u)} v_i + \mathbf{c}_i^{(u)} \cdot \nabla v_i \right), \quad (8)$$

so the material’s whole residual is one form whose coefficients are the componentwise sums. A residual unit publishes its coefficients directly. An energy unit publishes the flux against its staged variable, and the change of basis is the framework’s own statement rather than an assumption: the staged variable carries the expression defining it, $F = I + \nabla u$ for a deformation gradient, so

$$\mathbf{c}_i = \left. \frac{\partial W}{\partial F_i} \right|_{F=I+\nabla u} = \frac{\partial W}{\partial (\nabla u)_i}, \quad (9)$$

which was checked entry by entry against differentiating W directly and agrees exactly in rational arithmetic.

This is what makes the sampled merit possible at all, and it is worth something on its own. The combined form is emitted as one ordinary residual unit, so one traversal computes what the two computed between them. Measured on HEX8 at 72 threads on a Grace node, against the two units run the way the operator runs them:

elements	dofs	two units [MElem/s]	combined [MElem/s]	ratio
32,768	107,811	54.2	88.8	0.610
110,592	352,947	60.6	95.8	0.632
262,144	823,875	61.4	96.1	0.639
512,000	1,594,323	62.5	97.9	0.639

Throughput flattens by 262,144 elements, so the last two rows are saturated: $1.57\times$ the element rate, 36% of the time removed, with the two residuals agreeing to 8×10^{-16} relative. The saving is the second traversal’s gather, geometry and trial-function accumulation — the same quantities Section 4 hoists out of the trial-step loop, here hoisted out of the unit loop.

The combination is refused where it would be a second path to an existing answer: a material already written as one unit *is* its own total residual. It is also refused for mixed-order systems, whose fields do not share a shape count, because the lowered components carry no record of which element each came from — a Taylor–Hood material keeps its kernels and falls back to assembling per trial step.

4 The affine structure that makes one pass exact

Fix an element e and a quadrature point ξ_q . The element kernel reads the state only through the value and the reference gradient of the interpolated field,

$$u_q^e(y) = \sum_{j=1}^{N_s} y_j^e \varphi_j(\xi_q), \quad \widehat{\nabla} u_q^e(y) = \sum_{j=1}^{N_s} y_j^e \widehat{\nabla} \varphi_j(\xi_q), \quad y^e = P_e y. \quad (10)$$

Both maps $y \mapsto u_q^e$ and $y \mapsto \widehat{\nabla} u_q^e$ are *linear*. With $y = x + \alpha h$ this gives, exactly and for every α ,

$$u_q^e(x + \alpha h) = u_q^e(x) + \alpha u_q^e(h), \quad \widehat{\nabla} u_q^e(x + \alpha h) = \widehat{\nabla} u_q^e(x) + \alpha \widehat{\nabla} u_q^e(h). \quad (11)$$

The right-hand sides of (11) are two accumulations over the N_s shape functions, independent of α . The left-hand side is what the constitutive law consumes. So the sum over shape functions in (10) — the expensive part, $O(N_s d)$ per quadrature point per field — is performed *twice* in total, not $2m$ times, and each trial step recovers its own state with d fused multiply-adds.

The same argument carries through the geometric map, which is also linear:

$$\nabla u_q^e = J_e^{-\top} \widehat{\nabla} u_q^e \implies \nabla u_q^e(x + \alpha h) = \nabla u_q^e(x) + \alpha \nabla u_q^e(h), \quad (12)$$

so the combination may equivalently be formed in reference space before the map or in physical space after it. Nothing downstream of ∇u_q^e is affine — the first Piola–Kirchhoff stress $P(\nabla u)$ of a Mooney–Rivlin solid is not — and nothing downstream is hoisted. This is a redistribution of exact arithmetic, not a truncation: $\phi(\alpha_k)$ is the same number the per-step assembly computes, up to the reassociation of a sum.

5 The kernel: one node at a time, vectorized twice

The work unit is a single owned node n , not an element block. Write $\mathcal{E}(n)$ for the elements incident on n , which the node-to-element graph supplies directly. Then (3) restricted to that node is a sum over $\mathcal{E}(n)$ and nothing else,

$$[R_M(y)]_n = \sum_{e \in \mathcal{E}(n)} [r_M^e(P_e y)]_n, \quad (13)$$

so the node’s value is complete the moment that loop closes. It can be squared on the spot, and *no accumulator array exists at all*: the m residuals of one node are mN_c numbers living in registers, seeded from the accumulator g of (6) and discarded as soon as they are squared into the thread’s m -slot buffer. The patch-local node storage an element-centric traversal needs — $m|\mathcal{N}_p|N_c$ words, the one structure that grew with m — is gone.

Two vectorization axes, one after the other

The lanes are filled from within one node’s patch: the elements of $\mathcal{E}(n)$, and the trial steps $\alpha_1, \dots, \alpha_m$. What is deliberately *not* done is to fill lanes by processing several patches at once; the lanes belong to one node.

The two axes are two *sequential* vector loops, never one inside the other. Nesting them is not an arrangement the hardware has: a lane loop containing another loop is no longer innermost and does not vectorize at all, which is the same measured fact the stepped objective already records from the other direction. So the first loop is vectorized over the incident elements and computes everything independent of the trial step — the gathers, the geometry, the reference gradients of the state and of the direction, the physical gradient of the fixed basis function, and $w_q \det J_e$ — writing them to per-element buffers. The second loop reads those buffers and does only what the trial step changes: the combination, the constitutive law, and the contraction.

The orientation that pays for it

A node sits at a different local index in each of its incident elements, so the basis function paired with it — the test function of (8) — would differ from lane to lane, and nothing about it could be hoisted. Each element is therefore presented with its local numbering permuted so that n is always local node 0. Then φ_0 is the same basis function in every lane and at every trial step, and everything belonging to it leaves both loops: for a low-order simplex $\widehat{\nabla}\varphi_0$ is a constant the generator emits as a literal rather than a table read, and $J_e^{-\top}\widehat{\nabla}\varphi_0$ is computed in the first loop and merely read in the second.

The permutation is not free to be any relabelling. It must preserve the sign of $\det J_e$, or the element's volume changes sign and its contribution is subtracted; on a simplex that restricts it to the even permutations carrying the chosen vertex to position 0, which exist for every vertex.

Algorithm 1 Residual merit at m trial steps: one node, two vector loops

```

1:  $s \leftarrow 0 \in \mathbb{R}^m$  ▷ thread-private, the only output
2:  $\widehat{\nabla}\varphi_0$  ▷ a literal: same for every node, lane and step
3: for each owned node  $n$  (threads) do
4:    $\rho_k \leftarrow g_n \in \mathbb{R}^{N_c}$ ,  $k = 1, \dots, m$  ▷ registers, not memory
   loop 1 — lanes are the elements of  $\mathcal{E}(n)$ 
5:   for  $e \in \mathcal{E}(n)$  do
6:     present  $e$  with  $n$  at local 0, by an even permutation
7:     gather  $x^e, h^e$ ; form  $\det J_e$ ,  $\text{adj } J_e$ 
8:     for  $q = 1, \dots, N_q$  do
9:        $G_q^e \leftarrow \widehat{\nabla}u_q$ ,  $H_q^e \leftarrow \widehat{\nabla}w_q$ 
10:       $T_q^e \leftarrow J_e^{-\top}\widehat{\nabla}\varphi_0$ ,  $\omega_q^e \leftarrow w_q \det J_e$ 
11:    end for
12:  end for
  loop 2 — lanes are the trial steps  $\alpha_k$ 
13:  for  $k = 1, \dots, m$  do
14:    for  $e \in \mathcal{E}(n)$ ,  $q = 1, \dots, N_q$  do
15:       $\rho_k += \omega_q^e P(J_e^{-\top}(G_q^e + \alpha_k H_q^e)) T_q^e$ 
16:    end for
17:     $s_k += \frac{1}{2} \|\rho_k\|^2$  ▷ complete by (13)
18:  end for
19: end for
20:  $\phi(\alpha_k) \leftarrow \sum_{\text{threads}} s_k$ 

```

The second loop's lanes are the trial steps, and the reason is that m is ours to choose. The objection that would otherwise count against that axis — a runtime count shorter than the vector width, leaving lanes idle — does not apply to a quantity the line search sets: m is rounded to the hardware's vector length, or a multiple of it. The lanes are then uniform, always full and identical for every node, whereas the number of elements incident on a node is a property of the mesh, varies from node to node, and leaves a ragged tail no choice of ours can remove.

Sampling more step lengths than a classical search would ask for is close to free in this arrangement, because the extra samples occupy lanes that would otherwise idle. Rounding m up to the vector width buys a finer picture of ϕ along the direction at no cost, which is a pleasant inversion of the usual trade.

The alternatives — lanes over the incident elements with the steps serial, and lanes over the flattened element-by-step product — are not built. Competing loop arrangements are normally settled here by measurement rather than by argument, and this one is the exception because the comparison has no contested regime: the step count is chosen and the incident count is not,

so the mesh-determined axis is never the better-behaved one and building it would confirm that at the price of a second kernel to keep correct.

What the shape costs is that an element is visited once per node it carries, so the constitutive evaluation $P(\nabla u)$ is performed about N_s times where an element-centric pass performs it once and contracts it against every test function. That cost is measured rather than argued. On TET4 Mooney–Rivlin Kelvin–Voigt at 768,000 elements on one Grace node at 72 threads, against the implementation that assembles the whole residual once per trial step and fitted to $T(m) = a + bm$:

arm	a [s]	b [s]	$a/(a+b)$	$T(12)$ [s]
assemble per step	2.33×10^{-4}	3.01×10^{-3}	7.2%	3.63×10^{-2}
patch, node-centric	5.60×10^{-3}	1.40×10^{-3}	80.0%	2.22×10^{-2}

The step-independent fraction rises from 7% to 80%, which is the quantity this work exists to move and is past the 57% an already-sampled energy kernel of comparable constitutive cost reaches. At twelve trial steps the patch kernel is $1.63\times$ faster.

The redundancy is in the same numbers, and it is the honest half of the result: at a *single* step the patch kernel is $2.16\times$ slower, which is what visiting each element once per node costs. That it is 2.16 rather than $N_s = 4$ is the scatter and the accumulator traffic it no longer pays. The two cross between three and four trial steps, so the arrangement is a loss for a search sampling one or two points and a gain for anything sampling more — the regime the interface was built for.

6 Cost model

Write C_{gath} for the element gather and geometry, C_{acc} for one trial-function accumulation (10), C_{geo} for the geometric map, C_{mat} for the constitutive evaluation and C_{con} for the test-function contraction, all per element. The per-trial-step implementation costs

$$T_{\text{naive}}(m) = m(C_{\text{gath}} + C_{\text{acc}} + C_{\text{geo}} + C_{\text{mat}} + C_{\text{con}}) + mT_{\mathcal{A}}, \quad (14)$$

where $T_{\mathcal{A}}$ is the assembly of the state-independent operators. The sampling kernel costs

$$T_{\text{samp}}(m) = C_{\text{gath}} + 2C_{\text{acc}} + m(C_{\text{geo}} + C_{\text{mat}} + C_{\text{con}}) + T_{\mathcal{A}}, \quad (15)$$

with $T_{\mathcal{A}}$ appearing once by (6). The asymptotic ratio is

$$\lim_{m \rightarrow \infty} \frac{T_{\text{samp}}(m)}{T_{\text{naive}}(m)} = \frac{C_{\text{geo}} + C_{\text{mat}} + C_{\text{con}}}{C_{\text{gath}} + C_{\text{acc}} + C_{\text{geo}} + C_{\text{mat}} + C_{\text{con}} + T_{\mathcal{A}}/|\mathcal{E}|}, \quad (16)$$

The memory side is the sharper of the two, and it is where the patch-local contraction earns its keep. Writing n for the number of degrees of freedom, the per-trial-step implementation forms a combined iterate, assembles a residual into an n -vector and reduces it, once per step: $\Theta(mn)$ words written and $\Theta(mn)$ read, none of it reused. The sampling kernel reads x , h and g — $3n$ words, independent of m — and writes m scalars per thread. Nothing in the sampling kernel grows with m except the mN_c registers holding one node’s residuals, which are live for the length of one node’s element loop. The arithmetic above is therefore the honest limit: nothing in the sampling kernel becomes bandwidth-bound as m grows.

and at $m = 1$ the two agree up to the one extra accumulation C_{acc} . The measurement that settles the design is the line-search sweep over $m \in \{1, 2, 4, 8, 12, 20\}$ reported as $T(m)/T(1)$. Measured at 512,000 elements on one Grace node at 72 threads with `OMP_PROC_BIND=true`, and fitted to $T(m) = a + bm$ to separate the step-independent work a from the per-step work b :

arm	a [s]	b [s]	$a/(a+b)$	$T(12)/T(1)$
linear elasticity, energy	1.67×10^{-3}	5.36×10^{-4}	75.7%	3.70
Mooney-Rivlin, energy	1.72×10^{-3}	1.29×10^{-3}	57.1%	5.75
Mooney-Rivlin KV, residual	5.90×10^{-4}	9.09×10^{-3}	6.1%	11.78

The last row is the target and the middle row is what a sampled kernel of comparable constitutive cost achieves: today 6% of a residual merit evaluation is step-independent, against 57% once the sharing is in place. The number to move is that fraction, not the ratio directly.

7 What is built, and what is not

The form-layer combination of Section 3 is in place: the material’s whole residual is emitted as one kernel, and it agrees with the two units the operator runs to 8×10^{-16} relative on 512,000 elements. That kernel is the one the sampled merit contracts, and the measurement above is its floor.

The sampling kernel itself is not built. What remains is the trial-step loop inside the element traversal, the patch-local accumulators of Section 5, and the contraction to m scalars. The traversal those need is the packed one, and it does not yet exist for this kernel: the residual emitter’s packed path is restricted to single-field systems and is inactive for every material in the tree, so the combined kernel currently has no packed variant. Extending it to vector fields is the next step and is a prerequisite rather than an optimization, because the patch is what makes the node sums complete and therefore what makes the contraction legal at all.

One case is refused rather than approximated. A patch is built within one mesh block, so a node on a block boundary has incident elements the patch never sees and would be squared while its sum is still partial — a merit that comes out too small with nothing to show for it. Inter-block patches are the general answer; until they exist the sampled path refuses a multi-block space, and the per-step implementation, which assembles through the ordinary operator and spans blocks like any other assembly, continues to serve it.

8 The interface

Two predicates and one entry point express the split (6) at the `Op` boundary:

- `energy_or_potential_based()` — whether this operator’s 0-form is a potential, hence summable with others. False for a residual that is the gradient of nothing.
- `contracts_residual_merit()` — whether this operator is M . A false answer is the promise $\partial R_a/\partial y \equiv 0$ that puts the operator in $\mathcal{A}$.
- `residual_merit_steps(x, h, nsteps, steps, accumulator, out)` — evaluates (7) for every k in one pass, with `accumulator` carrying g .

Because (5) admits only one contractor, `add_operator` keeps the contracting operator last and refuses a second one.